

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ
«ДНІПРОВСЬКА ПОЛІТЕХНІКА»

Факультет: Інформаційних технологій

Кафедра: Програмного забезпечення комп'ютерних систем
(повна назва кафедри)

КУРСОВА РОБОТА

з дисципліни: ОРГАНІЗАЦІЯ БАЗ ДАНИХ І БАЗ ЗНАНЬ
(назва дисципліни)

на тему: «Розробка автоматизованої інформаційно-пошукової системи
для онлайн магазину жіночого одягу»

Студента (ки) 2 курсу, групи 121-23з-1

спеціальності інженерія програмного
забезпечення

Марковської Д.С.
(прізвище та ініціали)

Керівник доцент, к.т.н., доцент

Кабак Л.В.
(посада, вчене звання, науковий ступінь, прізвище та ініціали)

Національна шкала _____

Кількість балів: _____ Оцінка: ECTS _____

(підпис) Кабак Л.В.
(прізвище та ініціали)

м. Дніпро - 2024 рік

Зміст:

Зміст:	2
Вступ	3
Мета	4
Предметна область	4
1. Створення концептуальної моделі предметної області	5
1.1 Сутності	5
1.2 Зв'язки	6
2. Робота з базою даних	9
2.1 Створення представлень (Views)	9
2.2 Створення тригерів для внесення даних з представлень у основні таблиці	10
2.3 Тригери для автоматичної генерації ключів	11
3. Створення програми для взаємодії користувача з базою даних	14
3.1. Мета та завдання програми	14
3.2. Вибір середовища розробки та технологій	14
3.3. Архітектура програми	15
3.4 Опис взаємодії з базою даних	19
3.5 Функціональні можливості програми	19
3.6 Обробка помилок та валідація даних	21
Висновки	22
Список використаних джерел	23
Додатки	25
Додаток 1 Запит для створення БД	25
Додаток 2 Лістинг програми	29

Вступ

База даних — це набір інформації, яка зберігається в цифровому вигляді на комп'ютері, на сервері або в хмарі. Бази даних можуть бути дуже простими або досить складними, залежно від структури та організації даних. Система бази даних, як правило, складається з самої інформації та системи керування базами даних (СУБД), що дозволяє користувачам легко отримувати доступ, оновлювати, аналізувати та керувати інформацією.

Реляційні бази даних займають центральне місце в сучасних інформаційних системах і програмуванні. Слово "реляційний" походить від англійського слова "relation", що означає "відношення" або "зв'язок". У контексті баз даних цей термін описує організацію даних у вигляді таблиць, де кожна таблиця представляє набір пов'язаних між собою елементів інформації. Основною характеристикою реляційної бази даних є можливість встановлення зв'язків між різними таблицями на основі спільних полів, що забезпечує гнучкість та ефективність у роботі з великими обсягами інформації.

Реляційні бази даних організовані за допомогою таблиць, що складаються з рядків і стовпців. Кожен рядок представляє окремий запис, а стовпці містять окремі атрибути цього запису. Така структура дозволяє легко додавати нові записи, змінювати наявні дані та вилучати їх без порушення цілісності всієї системи. Однією з головних переваг реляційних баз є можливість встановлювати зв'язки між таблицями за допомогою ключів — унікальних ідентифікаторів для кожного запису.

Для взаємодії з реляційними базами даних широко використовується структурована мова запитів SQL (Structured Query Language). Вона дозволяє виконувати операції створення, оновлення, видалення та отримання даних. Завдяки стандартизованим командам SQL, розробники можуть ефективно працювати з різними реляційними СУБД (системами управління базами даних), такими як MySQL, PostgreSQL або Oracle.

Важливість реляційних баз даних у сучасному програмуванні важко переоцінити. Вони забезпечують надійну та ефективну організацію даних для різноманітних застосунків: від електронної комерції до складних корпоративних систем управління. Завдяки можливості побудови складних зв'язків між даними, реляційні бази дозволяють створювати гнучкі та масштабовані рішення, що задовольняють потреби бізнесу в будь-якому секторі.

У цій роботі для реалізації бази даних використовується операційна система macOS. Оскільки macOS має певні обмеження у підтримці деяких СУБД, таких як Microsoft SQL Server та Oracle, було вирішено використати PostgreSQL — реляційну систему керування базами даних, яка відмінно функціонує на macOS і надає всі необхідні можливості для роботи з даними. Також, для розробки обрано мову програмування Python, оскільки вона легко інтегрується з PostgreSQL та не викликає конфліктів з операційною системою. Використання цієї комбінації технологій забезпечує надійність і зручність при розробці програмного забезпечення, дозволяючи уникнути потенційних помилок і конфліктів сумісності.

Мета

Метою даної курсової роботи є:

- поглиблення і закріплення теоретичних знань з дисципліни «Організація баз даних та знань»;
- набуття навичок створення концептуальної моделі предметної області;
- набуття навичок розробки ER-діаграми бази даних;
- набуття навичок розробки тригерів для генерації ключів;
- набуття практичних навичок у поєднанні бази даних із програмним застосуванням

Предметна область

В даній курсовій роботі описана розробка інформаційно-пошукової моделі бази даних та програмного забезпечення, пов'язаного з нею, для працівників онлайн магазину жіночого одягу, що працює по системі дропшипінг. Створена програма

в поєднанні з базою даних надасть працівнику магазину можливість перевірити наявність товару за його артикулом, знайти товари за потрібним кольором, розміром чи категорією, знайти заміри потрібної речі за артикулом моделі та зв'язатися з постачальником для оформлення замовлення.

1. Створення концептуальної моделі предметної області

Важливу роль в розробці програмних засобів займає аналіз предметної області, проектування системи, розробка схеми бази даних. Від цього етапу розробки залежить швидкодія застосування та коректність виведення результатів.

Для коректної роботи програми відповідно до завдання, створена база даних має містити сутності та зв'язки між ними, наведені нижче.

1.1 Сутності

1. Продукти (Products_OD):

- Артикул (Code)
- Назва (Product_Name)
- Опис (Description)
- Категорія (Categories_OD_Category_Name)
- Зображення (Image_URL)
- Ціна (Price)

2. Категорії (Categories_OD):

- Назва категорії (Category_Name)

3. Кольори (Colors_OD):

- Назва кольору (ColorName)

4. Розміри (Sizes_OD):

- Ідентифікатор розміру (Size_ID)
- Назва розміру (Size_Name)

5. Заміри (Measures_OD):

- Ідентифікатор замірів (Measure_ID)
- Артикул (Products_OD_Code)

- Фото замірів (Img_URL)

- Опис (Description)

6. Постачальники (Suppliers_OD):

- Код постачальника (Supplier_ID)

- Назва постачальника (Supplier_Name)

- Посилання (Supplier_URL)

7. Інвентаризація (Inventory_OD):

- Ідентифікатор обліку (Inventory_ID)

- Артикул (Products_OD_Code)

- Ідентифікатор розміру (Sizes_OD_Size_ID)

- Назва кольору (Colors_OD_Color_Name)

- Наявність на складі (Available)

- Bltynsabrfnjh gjcnfxfkmysrf (Suppliers_OD_Supplier_ID)

1.2 Зв'язки

Продукт -> Категорія (багато-до-одного): Кожен продукт належить до певної категорії.

Продукт -> Інвентар (один-до-багатьох): Кожен продукт має запис у таблиці інвентарю з різними розмірами, кольорами та наявністю.

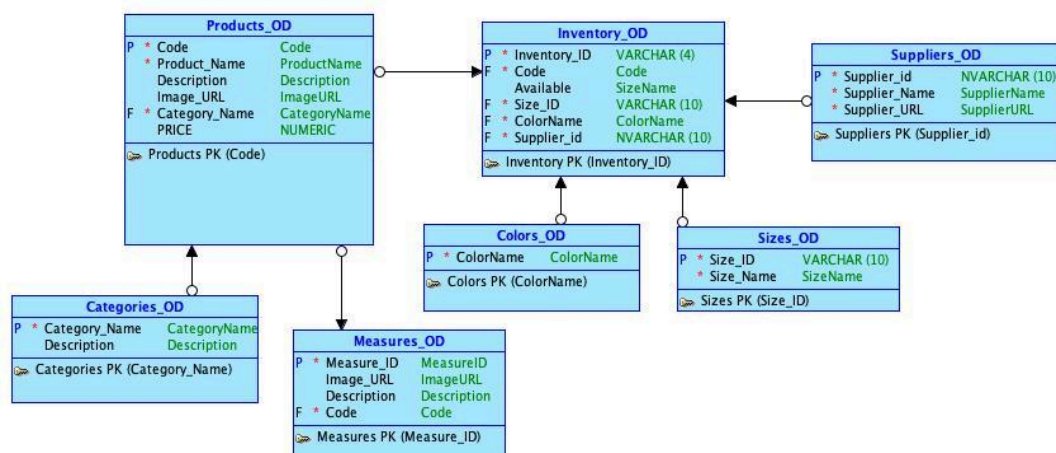
Продукт -> Заміри (один-до-одного): Кожному продукту відповідає один замір.

Інвентар -> Розміри (багато-до-одного): Інвентар має інформацію про доступні розміри для продукту.

Інвентар -> Кольори (багато-до-одного): Інвентар також містить інформацію про доступні кольори.

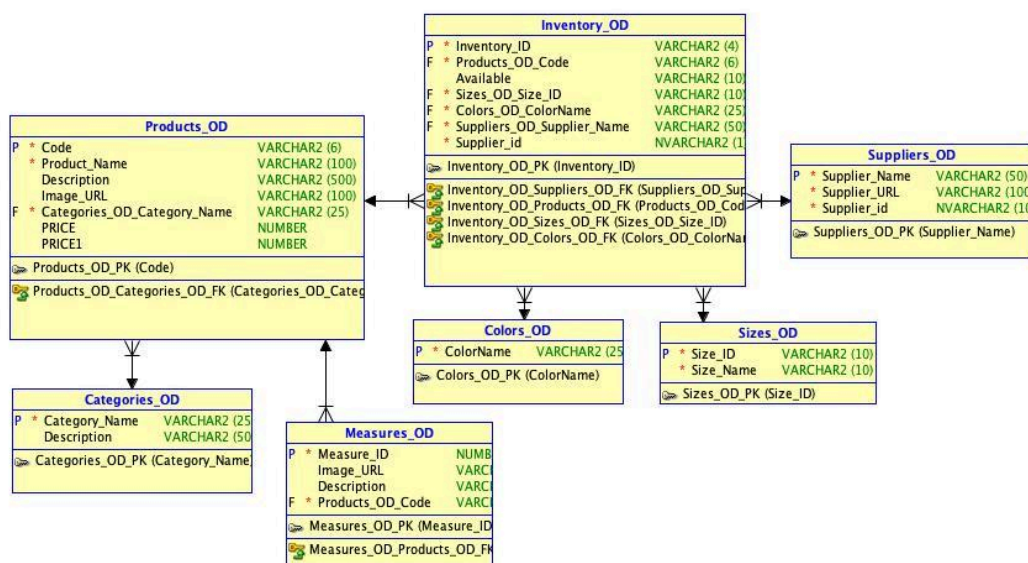
Інвентар -> Постачальник (багато-до-одного): Постачальники надають певні продукти з різними розмірами та кольорами.

На основі цих даних проектуємо логічну модель майбутньої бази даних за допомогою OracleDataModeler, хоча надалі у цій роботі буде використовуватись інша СУБД, цей додаток допоможе отримати готовий SQL запит для створення бази даних, який потрібно буде лише трохи змінити, бо у Oracle та PostgreSQL відрізняються назви типів даних.



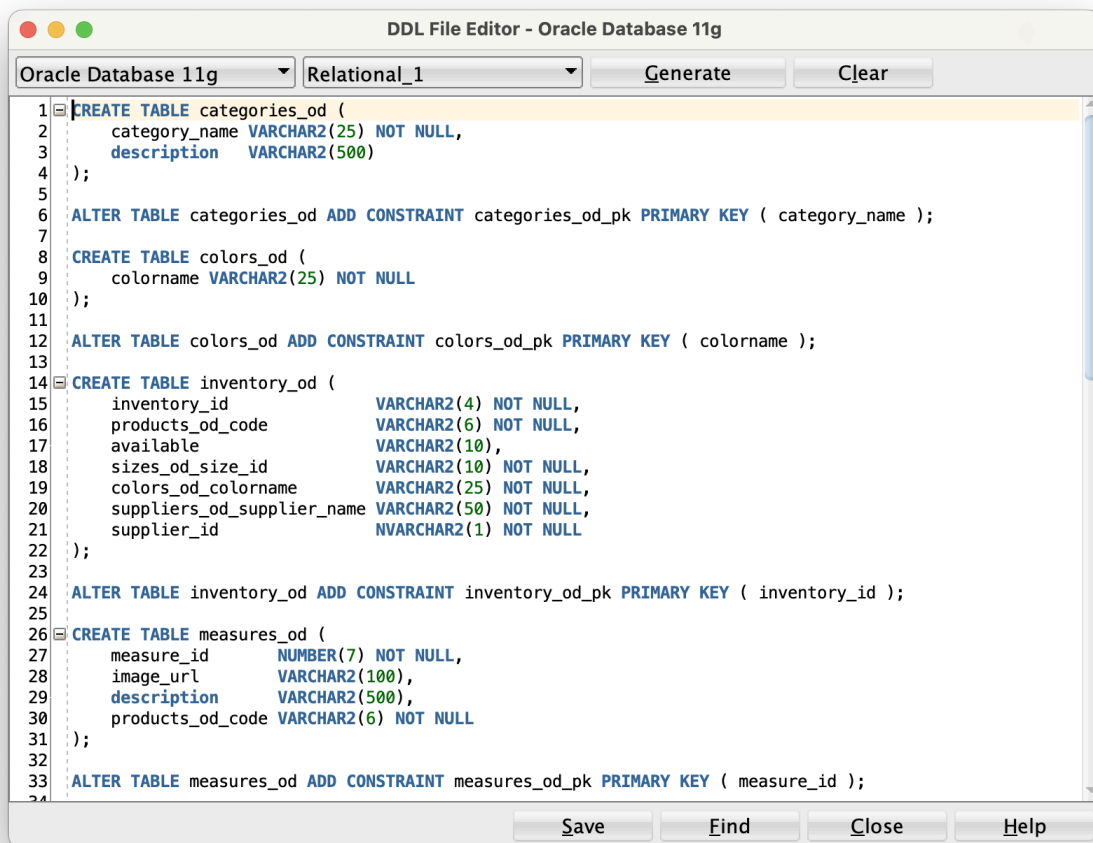
Мал. 1 Логічна модель бази даних

Після створення логічної моделі бази даних, можна досить просто конвертувати ці дані для створення ER-діаграми, у програмі OracleDataModeler. Після чого отримаємо модель бази даних, як показано на малюнку 2.



Мал. 2 ER-діаграма бази даних

Всі потрібні кроки для створення бази даних зроблено, тепер експортуємо дані у DDL файл, перед цим обираємо потрібну версію Oracle, від цього залежить синтаксис запитів, які генерує програма. Отримуємо готові SQL запити, які створять всі необхідні таблиці та зв'язки між ними.



Мал. 3 Згенерований запит для створення БД за ER-діаграмою

Так як цей запит створено під СУБД Oracle SQLDeveloper, а ми плануємо працювати на PostgreSQL у СУБДР TablePlus, маємо змінити типи даних:

- NVARCHAR2 та VARCHAR2 на VARCHAR;
- NUMBER на NUMERIC

Відредаговані запити зберігаємо у SQL файл та створюємо за допомогою нього базу даних з усіма таблицями. Повний код для створення таблиць можна переглянути у додатку 1 «Запит для створення БД».

2. Робота з базою даних

Після створення таблиць бази даних та встановлення зв'язків між ними, наступні кроки включають створення представлень (views) і тригерів для автоматизації процесів взаємодії з базою даних.

2.1 Створення представлень (Views)

Представлення (view) – це віртуальні таблиці, які можуть об'єднувати інформацію з різних таблиць та фільтрувати її для зручності користувача. Основною метою є спрощення взаємодії користувачів із базою даних, приховування складної структури та забезпечення доступу лише до необхідної інформації.

Представлення для відображення доступних товарів може включати інформацію з таблиць products_od, inventory_od, sizes_od, colors_od і suppliers_od. Це дозволяє користувачеві бачити всі дані про товари, доступні в різних кольорах, розмірах та від різних постачальників. SQL запит виглядатиме так:

```
CREATE VIEW available_products AS
```

```
SELECT p.code, p.product_name, p.description, p.price, i.available, s.size_name,  
c.colourname, sup.supplier_name, sup.supplier_url
```

```
FROM products_od p
```

```
JOIN inventory_od i ON p.code = i.products_od_code
```

```
JOIN sizes_od s ON i.sizes_od_size_id = s.size_id
```

```
JOIN colors_od c ON i.colors_od_colourname = c.colourname
```

```
JOIN suppliers_od sup ON i.suppliers_od_supplier_id = sup.supplier_id
```

```
WHERE i.available > 0;
```

2.2 Створення тригерів для внесення даних з представлень у основні таблиці

Тригери забезпечують автоматичне виконання певної логіки під час додавання, оновлення або видалення даних через представлення. Наприклад, після внесення нового запису через представлення, тригер може автоматично зберігати інформацію в основну таблицю.

Наприклад, якщо користувач додає новий товар через представлення, можна створити тригер, який автоматично вставить ці дані у відповідні таблиці. Запит буде виглядати так:

```
CREATE OR REPLACE FUNCTION insert_product_trigger()
```

```
RETURNS TRIGGER AS $$
```

```
BEGIN
```

```
    INSERT INTO products_od (code, product_name, description, price)
```

```
    VALUES (NEW.code, NEW.product_name, NEW.description, NEW.price);
```

```
    INSERT INTO inventory_od (inventory_id, products_od_code, available,  
    sizes_od_size_id, colors_od_colorname, suppliers_od_supplier_name)
```

```
    VALUES (NEW.inventory_id, NEW.code, NEW.available, NEW.sizes_od_size_id,  
    NEW.colors_od_colorname, NEW.suppliers_od_supplier_name);
```

```
    RETURN NEW;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER insert_product
```

```
INSTEAD OF INSERT ON available_products
```

```
FOR EACH ROW
```

```
EXECUTE FUNCTION insert_product_trigger();
```

2.3 Тригери для автоматичної генерації ключів

Автоматична генерація ключів спрощує процес введення даних для користувачів, оскільки унікальні ідентифікатори (ключі) будуть створюватися автоматично без необхідності ручного введення.

Для цього можна створити тригер, який автоматично генерує унікальний ключ для поля `inventory_id`, наприклад, використовуючи послідовність. Запит виглядатиме так:

```
CREATE SEQUENCE inventory_seq START 1;
```

```
CREATE OR REPLACE FUNCTION generate_inventory_id()
```

```
RETURNS TRIGGER AS $$
```

```
BEGIN
```

```
    NEW.inventory_id := 'INV' || LPAD(NEXTVAL('inventory_seq')::text, 4, '0');
```

```
    RETURN NEW;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER inventory_id_generator
```

```
BEFORE INSERT ON inventory_od  
  
FOR EACH ROW  
  
EXECUTE FUNCTION generate_inventory_id();
```

Також розробимо генерацію ключів для таблиці suppliers_od та її первинного ключа supplier_id:

```
CREATE SEQUENCE suppliers_seq START 1;  
  
CREATE OR REPLACE FUNCTION generate_supplier_id()  
  
RETURNS TRIGGER AS $$  
  
BEGIN  
  
    NEW.supplier_id := 'SUP' || LPAD(NEXTVAL('seppplier_seq')::text, 4, '0');  
  
    RETURN NEW;  
  
END;  
  
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER supplier_id_generator  
  
BEFORE INSERT ON suppliers_od  
  
FOR EACH ROW  
  
EXECUTE FUNCTION generate_supplier_id();
```

Останньою розробимо послідовність для генерації ключа для таблиці measures_od:

```
CREATE SEQUENCE mesures_seq START 1;
```

```
CREATE OR REPLACE FUNCTION generate_measure_id()
```

```
RETURNS TRIGGER AS $$
```

```
BEGIN
```

```
    NEW.measure_id := 'MES' || LPAD(NEXTVAL('mesures_seq')::text, 4, '0');
```

```
    RETURN NEW;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER measure_id_generator
```

```
BEFORE INSERT ON measures_od
```

```
FOR EACH ROW
```

```
EXECUTE FUNCTION generate_measure_id();
```

Для таблиці products_od розробляти генерацію ключа не потрібно, бо це буде унікальний номер який відповідає моделі речі в базі постачальника, тож його співробітники магазину, вноситимуть вручну, через створену програму.

3. Створення програми для взаємодії користувача з базою даних

3.1 Мета та завдання програми

У сучасних умовах бізнесу ефективне управління товарами є критично важливим для успіху будь-якого магазину. Розроблена програма має на меті створити зручний інтерфейс для взаємодії користувача з базою даних магазину одягу. Основними завданнями програми є забезпечення можливості швидкого і простого пошуку товарів за різними критеріями, такими як артикул, категорія, колір та розмір. Користувач повинен мати змогу отримати актуальну інформацію про наявність товарів, переглядати опис продуктів, а також здійснювати додавання нових товарів до бази даних. Окрім того, програма повинна забезпечити функціонал для формування звітів у двох найпоширеніших форматах, таких як Word та Excel, що дозволяє генерувати потрібну документацію безпосередньо з інтерфейсу програми.

Також програма має на меті спростити процес введення даних. Завдяки розробці зручних форм для введення інформації, співробітники магазину зможуть швидко додавати нові товари, заповнюючи лише необхідні поля. Це забезпечить ефективне управління товарним асортиментом та покращить обслуговування клієнтів. Загалом, основною метою програми є оптимізація робочих процесів у магазині, зменшення часу на виконання рутинних завдань та підвищення задоволеності користувачів завдяки зручному та інтуїтивно зрозумілому інтерфейсу.

3.2 Вибір середовища розробки та технологій

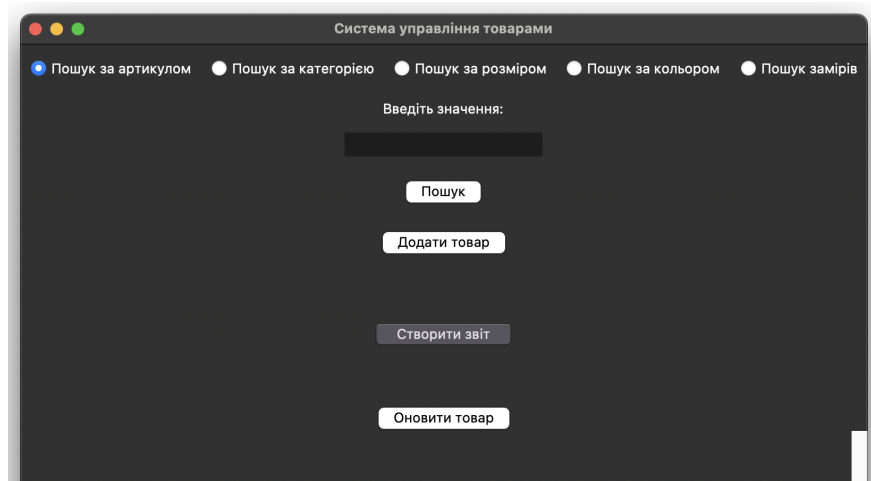
Вибір середовища розробки та технологій для створення програми є одним з ключових етапів, оскільки це визначає не лише зручність роботи для розробника, а й загальну продуктивність програми. Для розробки програми було обрано середовище PyCharm, яке є одним із найпопулярніших IDE (Integrated Development Environment — інтегроване середовище розробки, яке об'єднує в одному програмному пакеті всі необхідні інструменти для створення, тестування

та налагодження програмного забезпечення. IDE забезпечує зручний інтерфейс для програмістів, що дозволяє їм ефективно писати код, організовувати проекти, виконувати компіляцію та налагодження програм) для Python. PyCharm забезпечує зручний інтерфейс, потужні інструменти для налагодження та підтримує інтеграцію з системами контролю версій, що спрощує процес командної роботи над проектом. Компанія надає свої продукти для студентів безкоштовно, тому автор був радий цим скористатися.

Для управління базою даних використано PostgreSQL, яка відзначається високою продуктивністю та надійністю. PostgreSQL підтримує складні запити, транзакції та масштабованість, що робить її ідеальним вибором для проектів, що передбачають обробку великих обсягів даних. Крім того, програма використовує бібліотеку Tkinter для створення графічного інтерфейсу користувача. Tkinter є стандартною бібліотекою для розробки GUI (Graphical User Interface — графічний інтерфейс користувача, який дозволяє взаємодіяти з комп'ютерними програмами за допомогою графічних елементів, таких як вікна, кнопки, меню та інші візуальні компоненти) на Python і надає широкий спектр віджетів для створення інтерактивних вікон, що дозволяє реалізувати зручний та функціональний інтерфейс.

3.3 Архітектура програми

Архітектура програми розроблена з урахуванням зручності користувачів та масштабованості. Програма має модульну структуру, де кожен компонент відповідає за окрему функціональність. Основне вікно програми містить набір кнопок, кожна з яких виконує певну дію, пов'язану з взаємодією з базою даних. Наприклад, кнопки для пошуку товарів за артикулом, категорією, кольором або замірами, а також кнопка для додавання нового товару.



Мал. 4 Головне вікно програми

При натисканні на кнопку «Додати товар» та «Оновити товар» відкривається нове вікно, яке містить відповідні поля для введення даних.

Мал. 5.1 Вікно додавання товару

Додати товар

Оберіть категорію: ▼

Оберіть кольори:

- ☐ жовтий
- ☐ білий
- ☐ червоний
- ☐ фіолетовий
- ☐ голубий
- ☐ зелений
- ☐ рожевий
- ☐ молоко
- ☐ сірий
- ☐ шоколад
- ☐ синій
- ☐ чорний
- ☐ беж

Оберіть розміри:

- ☐ s-m
- ☐ m-l
- ☐ s-l
- ☐ s
- ☐ m
- ☐ l

Зберегти

Мал. 5.2 Вікно додавання товару

Оновити дані товару

Код товару:

Нова кількість у наявності:

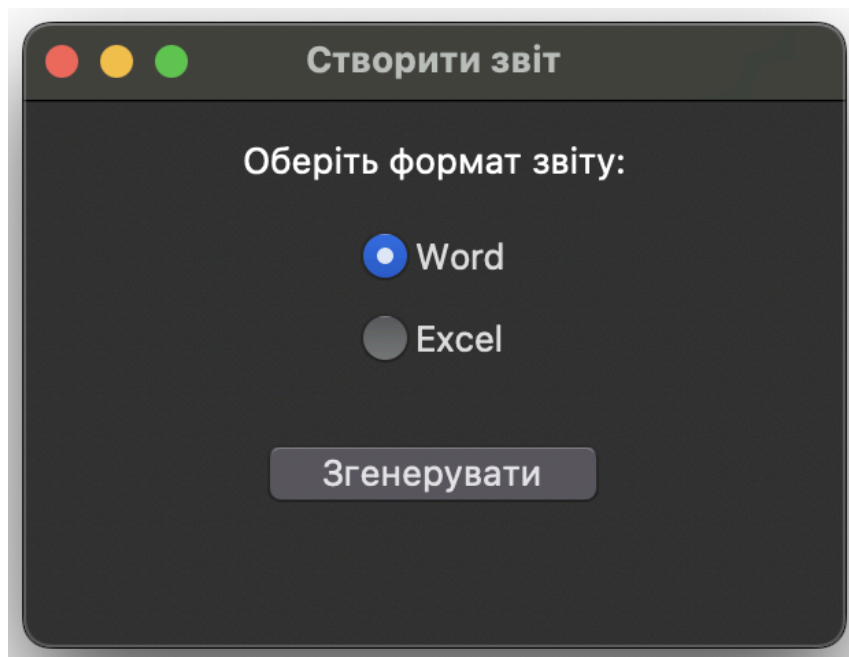
Новий розмір: ▼

Новий колір: ▼

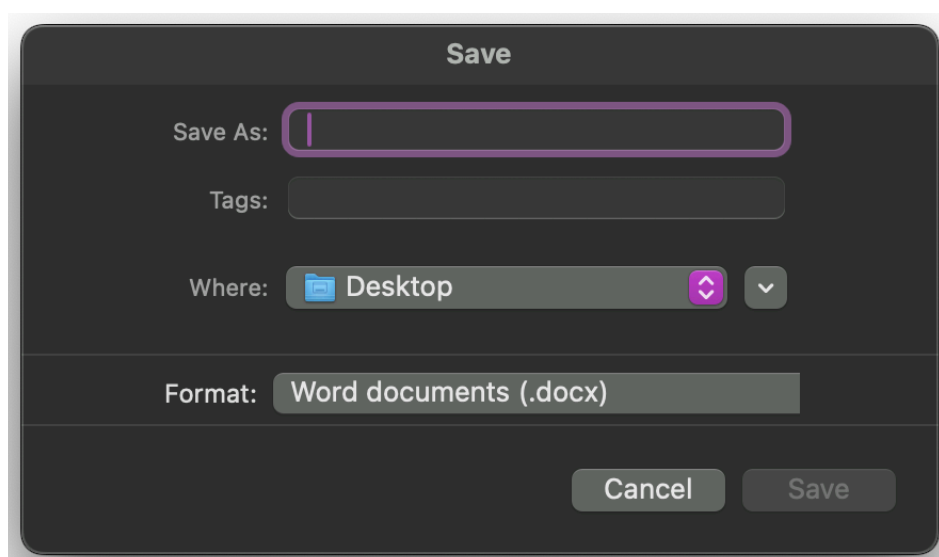
Оновити

Мал. 6 Вікно оновлення даних про товар

Інтерфейс програми спроектований таким чином, щоб користувач міг легко орієнтуватися та швидко виконувати необхідні дії. Програма також реалізує обробку подій, завдяки чому вона реагує на дії користувача у режимі реального часу. Це дозволяє зменшити затримки при виконанні запитів до бази даних та покращити загальний користувацький досвід. Також передбачено механізми для підтвердження дій користувача, що дозволяє запобігти випадковому видаленню або зміні важливих даних.



Мал. 7 Вікно вибору формату звіту



Мал. 8 Вікно вибору шляху завантаження звіту

3.4 Опис взаємодії з базою даних

Взаємодія програми з базою даних реалізована за допомогою бібліотеки `psycopg2`, що забезпечує зручний доступ до PostgreSQL. Після старту програми відбувається підключення до бази даних, яке здійснюється через функцію, що містить параметри з'єднання, такі як хост, порт, ім'я бази даних, ім'я користувача та пароль. Це дозволяє уникнути повторних підключень до бази даних у кожній функції, що значно підвищує ефективність роботи програми.

```
12 # Функція для підключення до бази даних
13 def connect_to_db():
14     try:
15         conn = psycopg2.connect(
16             dbname="postgres",
17             user="postgres",
18             password="3540576",
19             host="localhost",
20             port="5433"
21         )
22         return conn
23     except Exception as e:
24         messagebox.showerror( title: "Error", message: f"Error connecting to the database: {e}")
25         return None
```

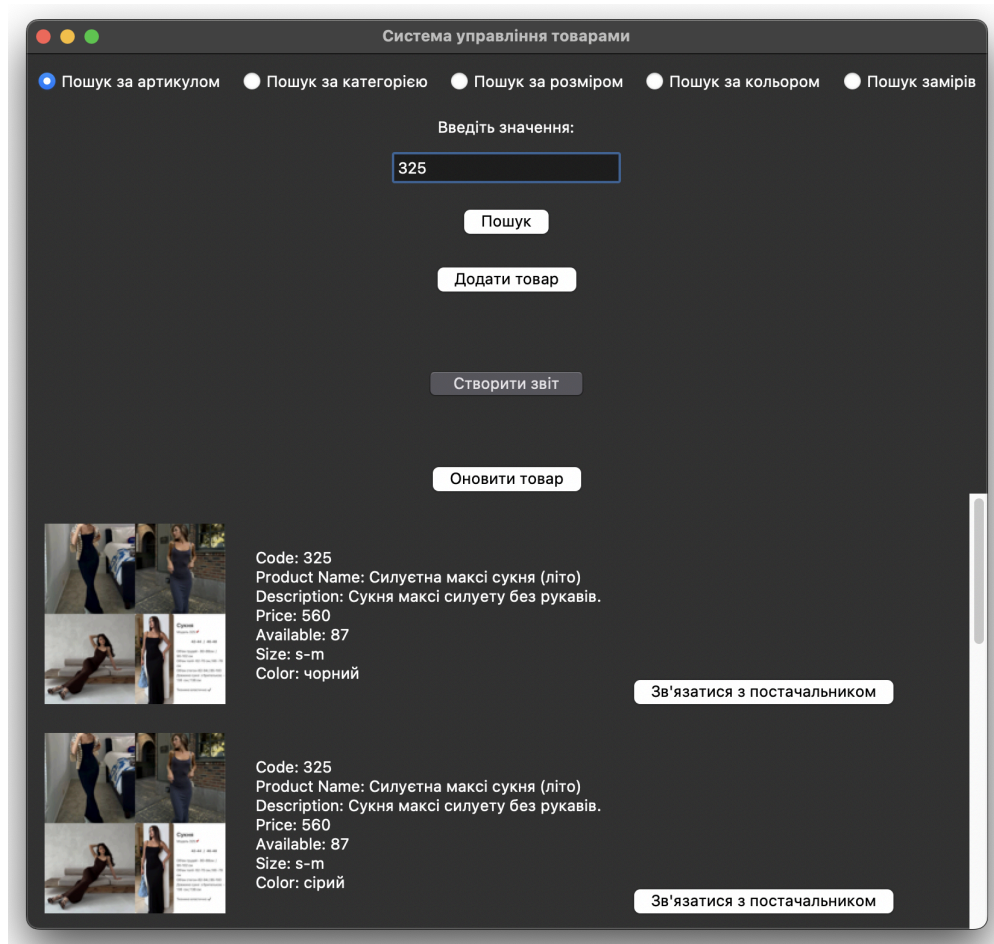
Мал. 9 Функція підключення до БД

Для отримання даних програма використовує SQL-запити типу `SELECT`, що дозволяють витягувати необхідну інформацію з таблиць бази даних, таких як `products_od`, `inventory_od`, `sizes_od`, та `suppliers_od`. При пошуку товарів за артикулом, наприклад, виконується запит, що повертає всі дані про відповідний товар, включаючи його наявність у різних розмірах та кольорах. Для додавання нових товарів до бази даних програма використовує запити типу `INSERT`, які дозволяють вводити нові записи у таблиці `products_od` та `inventory_od`. Це спрощує управління даними і дозволяє швидко оновлювати асортимент товарів.

3.5 Функціональні можливості програми

Програма надає користувачам кілька основних функцій, які полегшують взаємодію з базою даних. По-перше, реалізовано можливість пошуку товарів за артикулом, що дозволяє користувачу вводити код товару і швидко отримувати

детальну інформацію про нього. Програма виконуватиме запит до бази даних і відобразить усі доступні дані, включаючи фото товару, назву, його опис, ціну та наявність у різних кольорах та розмірах. Також реалізована кнопка «Зв'язатися з постачальником», яка відкриває посилання на постачальника у телеграмм.



Мал. 9 Результат пошуку за артикулом товару

По-друге, передбачено пошук за категорією товарів. Користувач може ввести категорію та почати пошук. Програма відобразить всі товари, що належать до обраної категорії, що забезпечить зручність у навігації та спростить процес вибору.

Окрім цього, програма має можливість додавання нових товарів. У вікні для додавання товару користувач може заповнити усі необхідні поля, а після натискання кнопки «Додати» дані будуть збережені у базі даних. Це дозволяє користувачеві легко оновлювати асортимент, не вимагаючи великих зусиль.

3.6 Обробка помилок та валідація даних

Обробка помилок та валідація даних є важливими аспектами програми, оскільки вони гарантують коректність і надійність введеної інформації. Програма реалізує кілька механізмів для забезпечення цього. Перш за все, перед внесенням нових даних у базу перевіряється, чи заповнені всі обов'язкові поля. Наприклад, якщо користувач намагається додати товар без вказівки артикулу або назви, програма відобразить повідомлення про помилку і не дозволить виконати операцію до виправлення помилки.

Крім того, реалізовано перевірку формату введених даних. Наприклад, для артикулу товару встановлено обмеження на довжину та символи, що дозволяє уникнути помилок при введенні. Якщо користувач вводить некоректні дані, програма надає зрозуміле повідомлення з рекомендацією виправити введену інформацію.

Також програма використовує механізми обробки виключень, щоб запобігти аварійному завершенню роботи в разі виникнення помилки, пов'язаної з підключенням до бази даних або виконанням SQL-запитів. Це робить програму більш стабільною і надійною в умовах різних непередбачених ситуацій.

Повний код програми, можна побачити у додатках «Лістинг програми».

Висновки

В рамках лабораторної роботи було успішно розроблено модель бази даних для магазину одягу за допомогою Oracle Data Modeler, що дозволило створити чітку структуру з усіма необхідними сутностями, такими як категорії товарів, кольори, постачальники, розміри, продукти, запаси та вимірювання. Ця модель стала основою для реалізації реляційної бази даних, що забезпечує ефективне зберігання та управління даними.

Створення бази даних у PostgreSQL дало змогу реалізувати всі заплановані зв'язки між сутностями, що забезпечує цілісність даних і оптимізує процеси їх обробки. Наявність тригерів для автоматичної генерації ключів та внесення інформації з представлень у основні таблиці спростила введення даних користувачами та підвищила ефективність роботи з базою даних.

Крім того, було розроблено програму для взаємодії з користувачем, яка дозволяє здійснювати пошук, перегляд, додавання та модерацію товарів у базу даних через інтуїтивно зрозумілий графічний інтерфейс. Використання Python та бібліотеки Tkinter забезпечило зручний та функціональний інтерфейс, що значно покращує користувацький досвід.

Завдяки інтеграції всіх компонентів – моделі бази даних, реальної бази даних та програми – було досягнуто високого рівня автоматизації процесів управління асортиментом у магазині одягу. Результати лабораторної роботи демонструють ефективність запропонованої архітектури системи, що відповідає сучасним вимогам до управління даними та взаємодії з користувачами.

У подальшій роботі планується впровадження нових функцій, таких як автоматизація звітності та вдосконалення процесів обліку, що дозволить підвищити ефективність та продуктивність роботи магазину, а також поліпшити обслуговування клієнтів. Також, в майбутньому ця база даних буде розширена, та використовуватиметься для заповнення сайту інтернет магазину.

Список використаних джерел

1. Date, C. J. (2004). *An Introduction to Database Systems* (8th ed.). Addison-Wesley.
2. Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.
3. Korth, H. F., & Silberschatz, A. (2010). *Database System Concepts* (6th ed.). McGraw-Hill.
4. Ramakrishnan, R., & Gehrke, J. (2014). *Database Management Systems* (3rd ed.). McGraw-Hill.
5. Beech, M. (2012). *Database Design and Implementation*. O'Reilly Media.
6. Nielson, J. (2011). *Usability Engineering*. Morgan Kaufmann.
7. van der Lans, R. (2010). *Introduction to NoSQL*. The NoSQL Database Revolution.
8. Shibilin, V. (2018). *Системи управління базами даних: теорія та практика*. К.: Видавництво "Молодь".
9. Бродський, В. Б. (2010). *Проектування баз даних*. К.: Видавництво "Книжковий дім".
10. Федоренко, В. (2017). *Моделювання баз даних у середовищі Oracle Data Modeler*. К.: Видавництво "Техніка".
11. Tkinter. (n.d.). Retrieved from <https://docs.python.org/3/library/tkinter.html>
12. PostgreSQL Documentation. (n.d.). Retrieved from <https://www.postgresql.org/docs/>
13. Python Software Foundation. (n.d.). Retrieved from <https://www.python.org/>
14. Oracle Data Modeler. (n.d.). Retrieved from <https://www.oracle.com/database/technologies/appdev/datamodeler.html>
15. Zhang, M. (2020). *A Comprehensive Guide to Database Management*. Retrieved from <https://www.databasejournal.com/>
16. Codd, E. F. (1990). *The Relational Model for Database Management*. Addison-Wesley.
17. Gray, J. (1983). *The Transaction Concept: Virtues and Limitations*. ACM SIGMOD Record, 13(2), 1-13.

18. Ambler, S. W. (2002). *The Object Primer: Agile Model-Driven Development*. Cambridge University Press.
19. Melton, J., & Simon, A. (2002). *SQL: 1999 - Understanding Relational Language Components*. Morgan Kaufmann.
20. Fowler, M. (2003). *Patterns of Enterprise Application Architecture*. Addison-Wesley.

Додатки

Додаток 1 Запит для створення БД

```
CREATE TABLE categories_od (  
    category_name VARCHAR(25) NOT NULL,  
    description VARCHAR(500)  
);
```

```
ALTER TABLE categories_od ADD CONSTRAINT categories_od_pk PRIMARY  
KEY (category_name);
```

```
CREATE TABLE colors_od (  
    colorname VARCHAR(25) NOT NULL  
);
```

```
ALTER TABLE colors_od ADD CONSTRAINT colors_od_pk PRIMARY KEY  
(colorname);
```

```
CREATE TABLE inventory_od (  
    inventory_id VARCHAR(10) NOT NULL,  
    products_od_code VARCHAR(6) NOT NULL,  
    available NUMERIC,
```

```
sizes_od_size_id      VARCHAR(10) NOT NULL,  
colors_od_coloname    VARCHAR(25) NOT NULL,  
suppliers_od_supplier_id VARCHAR(10) NOT NULL  
);
```

```
ALTER TABLE inventory_od ADD CONSTRAINT inventory_od_pk PRIMARY  
KEY (inventory_id);
```

```
CREATE TABLE measures_od (  
    measure_id    VARCHAR(10) NOT NULL,  
    image_url     VARCHAR(100),  
    description   VARCHAR(500),  
    products_od_code VARCHAR(6) NOT NULL  
);
```

```
ALTER TABLE measures_od ADD CONSTRAINT measures_od_pk PRIMARY  
KEY (measure_id);
```

```
CREATE TABLE products_od (  
    code          VARCHAR(6) NOT NULL,  
    product_name  VARCHAR(100) NOT NULL,
```

```
description          VARCHAR(500),  
  
image_url            VARCHAR(100),  
  
categories_od_category_name VARCHAR(25) NOT NULL,  
  
price                NUMERIC  
  
);
```

```
ALTER TABLE products_od ADD CONSTRAINT products_od_pk PRIMARY  
KEY (code);
```

```
CREATE TABLE sizes_od (  
  
    size_id  VARCHAR(10) NOT NULL,  
  
    size_name VARCHAR(10) NOT NULL  
  
);
```

```
ALTER TABLE sizes_od ADD CONSTRAINT sizes_od_pk PRIMARY KEY  
(size_id);
```

```
CREATE TABLE suppliers_od (  
  
    supplier_id  VARCHAR(10) NOT NULL  
  
    supplier_name VARCHAR(50) NOT NULL,  
  
    supplier_url  VARCHAR(100) NOT NULL
```

);

```
ALTER TABLE suppliers_od ADD CONSTRAINT suppliers_od_pk PRIMARY  
KEY (supplier_name);
```

```
ALTER TABLE inventory_od  
  
    ADD CONSTRAINT inventory_od_colors_od_fk FOREIGN KEY  
(colors_od_colormame)  
  
    REFERENCES colors_od (colormame);
```

```
ALTER TABLE inventory_od  
  
    ADD CONSTRAINT inventory_od_products_od_fk FOREIGN KEY  
(products_od_code)  
  
    REFERENCES products_od (code);
```

```
ALTER TABLE inventory_od  
  
    ADD CONSTRAINT inventory_od_sizes_od_fk FOREIGN KEY  
(sizes_od_size_id)  
  
    REFERENCES sizes_od (size_id);
```

```
ALTER TABLE inventory_od
```

```
ADD CONSTRAINT inventory_od_suppliers_od_fk FOREIGN KEY
(suppliers_od_supplier_id)
```

```
REFERENCES suppliers_od (supplier_id);
```

```
ALTER TABLE measures_od
```

```
ADD CONSTRAINT measures_od_products_od_fk FOREIGN KEY
(products_od_code)
```

```
REFERENCES products_od (code);
```

```
ALTER TABLE products_od
```

```
ADD CONSTRAINT products_od_categories_od_fk FOREIGN KEY
(categories_od_category_name)
```

```
REFERENCES categories_od (category_name);
```

Додаток 2 Лістинг програми

```
import psycopg2
from docx import Document
from openpyxl import Workbook
import tkinter as tk
from tkinter import *
from tkinter import ttk, filedialog, messagebox
from PIL import Image, ImageTk
import requests
from io import BytesIO
import platform
```

Функція для підключення до бази даних

```
def connect_to_db():
```

```
    try:
```

```
        conn = psycopg2.connect(
```

```
            dbname="postgres",
```

```
            user="postgres",
```

```
            password="3540576",
```

```
            host="localhost",
```

```
            port="5433"
```

```
        )
```

```
        return conn
```

```
    except Exception as e:
```

```
        messagebox.showerror("Error", f"Error connecting to the database: {e}")
```

```
    return None
```

Функція для отримання товару за кодом

```
def query_product_by_code(conn, code):
```

```
    try:
```

```
        with conn.cursor() as cursor:
```

```
            query = """
```

```
                SELECT p.code, p.product_name, p.description, p.price, i.available,
```

```
                s.size_name, c.colourname, p.image_url, i.suppliers_od_supplier_id, u.supplier_url,
```

```
                u.supplier_name
```

```
                FROM products_od p
```

```
                JOIN inventory_od i ON p.code = i.products_od_code
```

```
                JOIN sizes_od s ON i.sizes_od_size_id = s.size_id
```

```
                JOIN colors_od c ON i.colors_od_colourname = c.colourname
```

```
                JOIN suppliers_od u ON i.suppliers_od_supplier_id = u.supplier_id
```

```
                WHERE p.code = %s;
```

```
            """
```

```
            cursor.execute(query, (code,))
```

```

        result = cursor.fetchall()
        display_results(result)
    except Exception as e:
        messagebox.showerror("Error", f"Error querying product by code: {e}")

```

Функція для отримання товарів за категорією

```
def query_products_by_category(conn, category_name):
```

```
    try:
```

```
        with conn.cursor() as cursor:
```

```
            query = """
```

```
                SELECT p.code, p.product_name, p.description, p.price, i.available,
s.size_name, c.colourname, p.image_url, i.suppliers_od_supplier_id, u.supplier_url,
u.supplier_name
```

```
                FROM products_od p
```

```
                JOIN inventory_od i ON p.code = i.products_od_code
```

```
                JOIN sizes_od s ON i.sizes_od_size_id = s.size_id
```

```
                JOIN colors_od c ON i.colors_od_colourname = c.colourname
```

```
                JOIN suppliers_od u ON i.suppliers_od_supplier_id = u.supplier_id
```

```
                WHERE p.categories_od_category_name = %s;
```

```
            """
```

```
            cursor.execute(query, (category_name,))
```

```
            result = cursor.fetchall()
```

```
            display_results(result)
```

```
    except Exception as e:
```

```
        messagebox.showerror("Error", f"Error querying products by category: {e}")
```

Функція для отримання товарів за розміром

```
def query_products_by_size(conn, size_id):
```

```
    try:
```

```
        with conn.cursor() as cursor:
```

```
            query = """
```

```

        SELECT p.code, p.product_name, p.description, p.price, i.available,
s.size_name, c.colourname, p.image_url, i.suppliers_od_supplier_id, u.supplier_url,
u.supplier_name

```

```

        FROM products_od p

```

```

        JOIN inventory_od i ON p.code = i.products_od_code

```

```

        JOIN sizes_od s ON i.sizes_od_size_id = s.size_id

```

```

        JOIN colors_od c ON i.colors_od_colourname = c.colourname

```

```

        JOIN suppliers_od u ON i.suppliers_od_supplier_id = u.supplier_id

```

```

        WHERE i.sizes_od_size_id = %s;

```

```

        """

```

```

        cursor.execute(query, (size_id,))

```

```

        result = cursor.fetchall()

```

```

        display_results(result)

```

```

except Exception as e:

```

```

    messagebox.showerror("Error", f"Error querying products by size: {e}")

```

```

# Функція для отримання товарів за кольором

```

```

def query_products_by_color(conn, color_name):

```

```

    try:

```

```

        with conn.cursor() as cursor:

```

```

            query = """

```

```

                SELECT p.code, p.product_name, p.description, p.price, i.available,
s.size_name, c.colourname, p.image_url, i.suppliers_od_supplier_id, u.supplier_url,
u.supplier_name

```

```

                FROM products_od p

```

```

                JOIN inventory_od i ON p.code = i.products_od_code

```

```

                JOIN sizes_od s ON i.sizes_od_size_id = s.size_id

```

```

                JOIN colors_od c ON i.colors_od_colourname = c.colourname

```

```

                JOIN suppliers_od u ON i.suppliers_od_supplier_id = u.supplier_id

```

```

                WHERE c.colourname = %s;

```

```

            """

```

```

        cursor.execute(query, (color_name,))
        result = cursor.fetchall()
        display_results(result)
    except Exception as e:
        messagebox.showerror("Error", f"Error querying products by color: {e}")

```

Функція для отримання замірів за кодом товару

```
def query_measures_by_code(conn, code):
```

```
    try:
```

```
        with conn.cursor() as cursor:
```

```
            query = """
```

```
                SELECT m.image_url, m.description
```

```
                FROM measures_od m
```

```
                JOIN products_od p ON p.code = m.products_od_code
```

```
                WHERE p.code = %s;
```

```
            """
```

```
            cursor.execute(query, (code,))
```

```
            result = cursor.fetchone()
```

```
            display_measures(result)
```

```
    except Exception as e:
```

```
        messagebox.showerror("Error", f"Error querying measures by code: {e}")
```

Функція для відображення замірів (зображення або опис)

```
def display_measures(result):
```

```
    clear_results()
```

```
    if result:
```

```
        measure_info = f"Description: {result[1]}"
```

```
        measure_label = tk.Label(result_frame, text=measure_info, justify='left',
anchor='w')
```

```
        measure_label.pack(side=tk.RIGHT, padx=10)
```

```
        if result[0]: # image_url
```

```

try:
    response = requests.get(result[0])
    img_data = response.content
    image = Image.open(BytesIO(img_data))
    image.thumbnail((150, 150))
    photo = ImageTk.PhotoImage(image)
    img_label = tk.Label(result_frame, image=photo)
    img_label.image = photo # Зберігаємо посилання на зображення, щоб

```

уникнути його знищення

```

    img_label.pack(side=tk.LEFT, padx=10)
except Exception as e:
    img_label = tk.Label(result_frame, text="Error loading image", fg="red")
    img_label.pack(side=tk.LEFT, padx=10)
else:
    img_label = tk.Label(result_frame, text="No image available", fg="gray")
    img_label.pack(side=tk.LEFT, padx=10)
else:
    no_result_label = tk.Label(result_frame, text="No measures found.", fg="red")
    no_result_label.pack()

```

Функція для відображення результатів пошуку

```
def display_results(results):
```

```
    clear_results()
```

```
    if results:
```

```
        for row in results:
```

```
            info_frame = tk.Frame(result_frame)
```

```
            info_frame.pack(anchor='w', pady=10)
```

```
        # Налаштування ширини колонок
```

```
        info_frame.grid_columnconfigure(0, minsize=150) # ширина для картинки
```

```
info_frame.grid_columnconfigure(1, minsize=300) # ширина для
інформації про товар
```

```
# Відображення зображення
```

```
if row[7]: # image_url
```

```
try:
```

```
    response = requests.get(row[7])
```

```
    img_data = response.content
```

```
    image = Image.open(BytesIO(img_data))
```

```
    image.thumbnail((150, 150))
```

```
    photo = ImageTk.PhotoImage(image)
```

```
    img_label = tk.Label(info_frame, image=photo)
```

```
    img_label.image = photo
```

```
    img_label.grid(row=0, column=0, padx=10)
```

```
except Exception as e:
```

```
    img_label = tk.Label(info_frame, text="Error loading image", fg="red")
```

```
    img_label.grid(row=0, column=0, padx=10)
```

```
else:
```

```
    img_label = tk.Label(info_frame, text="No image available", fg="gray")
```

```
    img_label.grid(row=0, column=0, padx=10)
```

```
# Відображення інформації про продукт
```

```
product_info = f"Code: {row[0]}\nProduct Name: {row[1]}\nDescription:
{row[2]}\nPrice: {row[3]}\nAvailable: {row[4]}\nSize: {row[5]}\nColor: {row[6]}"
```

```
info_label = tk.Label(info_frame, text=product_info, justify='left', anchor='w')
```

```
info_label.grid(row=0, column=1, padx=10)
```

```
# Кнопка "Зв'язатися з постачальником"
```

```
supplier_button = tk.Button(info_frame, text="Зв'язатися з
постачальником", command=lambda url=row[9]: open_supplier_link(url))
```

```
supplier_button.grid(row=0, column=2, padx=10, sticky='se')
```

else:

```
no_result_label = tk.Label(result_frame, text="No results found.", fg="red")
no_result_label.pack()
```

Відкриття посилання на постачальника

```
def open_supplier_link(url):
```

```
    import webbrowser
```

```
    webbrowser.open(url)
```

Очищення результатів

```
def clear_results():
```

```
    for widget in result_frame.winfo_children():
```

```
        widget.destroy()
```

Функція для обробки прокрутки миші/тачпада

```
def on_mouse_wheel(event):
```

```
    if platform.system() == "Darwin": # macOS
```

```
        scroll_canvas.yview_scroll(int(-1 * event.delta), "units")
```

```
    else:
```

```
        scroll_canvas.yview_scroll(int(-1 * event.delta / 120), "units")
```

Функція для обробки вибору користувача

```
def search():
```

```
    conn = connect_to_db()
```

```
    if conn:
```

```
        search_value = input_field.get()
```

```
        if search_by.get() == "code":
```

```
            query_product_by_code(conn, search_value)
```

```
        elif search_by.get() == "category":
```

```
            query_products_by_category(conn, search_value)
```

```
        elif search_by.get() == "size":
```

```

        query_products_by_size(conn, search_value)
    elif search_by.get() == "color":
        query_products_by_color(conn, search_value)
    elif search_by.get() == "measures":
        query_measures_by_code(conn, search_value)
    conn.close()

```

Функція для отримання кольорів

```

def get_colors(conn):
    try:
        with conn.cursor() as cursor:
            cursor.execute("SELECT colormame FROM colors_od;")
            return [row[0] for row in cursor.fetchall()]
    except Exception as e:
        messagebox.showerror("Error", f"Error fetching colors: {e}")
        return []

```

Функція для отримання розмірів

```

def get_sizes(conn):
    try:
        with conn.cursor() as cursor:
            cursor.execute("SELECT size_name FROM sizes_od;")
            return [row[0] for row in cursor.fetchall()]
    except Exception as e:
        messagebox.showerror("Error", f"Error fetching sizes: {e}")
        return []

```

Функція для отримання всіх доступних категорій

```

def get_categories(conn):
    try:
        with conn.cursor() as cursor:

```

```

        cursor.execute("SELECT category_name FROM categories_od;")
        return [row[0] for row in cursor.fetchall()]
except Exception as e:
    messagebox.showerror("Error", f"Error fetching categories: {e}")
return []

```

Функція для отримання постачальників

```

def get_suppliers(conn):
    try:
        with conn.cursor() as cursor:
            cursor.execute("SELECT supplier_id, supplier_name FROM suppliers_od;")
            return cursor.fetchall() # Повертаємо список кортежів (id, name)
    except Exception as e:
        messagebox.showerror("Error", f"Error fetching suppliers: {e}")
        return []

```

Функція для вставки тексту з буфера обміну

```

def paste_from_clipboard(entry_widget):
    try:
        # Отримання тексту з буфера обміну
        clipboard_text = root.clipboard_get()
        entry_widget.delete(0, tk.END) # Очищення поточного тексту
        entry_widget.insert(0, clipboard_text) # Вставка тексту
    except tk.TclError:
        messagebox.showwarning("Увага", "Буфер обміну порожній.")

```

Функція для обробки натискання клавіші

```

def on_key_press(event, entry_widget):
    if (event.state & 0x04) and event.keysym == 'v': # Cmd або Ctrl + V
        paste_from_clipboard(entry_widget)

```

```

# Функція для створення контекстного меню
def show_context_menu(event, entry_widget):
    context_menu = tk.Menu(root, tearoff=0)
        context_menu.add_command(label="Вставити",    command=lambda:
paste_from_clipboard(entry_widget))
    context_menu.post(event.x_root, event.y_root)

# Функція для відкриття вікна додавання товару
def open_add_product_window():
    add_product_window = tk.Toplevel(root)
    add_product_window.title("Додати товар")
    add_product_window.geometry("500x600") # Фіксовані розміри вікна

# Створення полотна та прокрутки
canvas = tk.Canvas(add_product_window)
        scrollbar    =    tk.Scrollbar(add_product_window,    orient="vertical",
command=canvas.yview)
    scrollable_frame = tk.Frame(canvas)
    scrollable_frame.bind(
        "<Configure>",
        lambda e: canvas.configure(scrollregion=canvas.bbox("all"))
    )
    canvas.create_window((0, 0), window=scrollable_frame, anchor="nw")
    canvas.configure(yscrollcommand=scrollbar.set)
    canvas.pack(side="left", fill="both", expand=True)
    scrollbar.pack(side="right", fill="y")
# Прокрутка вікна додавання товару
def on_mouse_wheel_add_window(event):
    canvas.yview_scroll(int(-1 * (event.delta / 120)), "units")

# Прив'язка прокрутки до тачпада/колеса миші

```

```

add_product_window.bind_all("<MouseWheel>", on_mouse_wheel_add_window)
# Для Windows та macOS
add_product_window.bind_all("<Button-4>", on_mouse_wheel_add_window) #
Для Linux
add_product_window.bind_all("<Button-5>", on_mouse_wheel_add_window) #
Для Linux

# Поля для введення даних про товар
tk.Label(scrollable_frame, text="Код товару:").grid(row=0, column=0, padx=10,
pady=10)
product_code_entry = tk.Entry(scrollable_frame)
product_code_entry.grid(row=0, column=1, padx=10, pady=10)
product_code_entry.bind("<KeyPress>", lambda event: on_key_press(event,
product_code_entry))
product_code_entry.bind("<Button-3>", lambda event: show_context_menu(event,
product_code_entry))
tk.Label(scrollable_frame, text="Назва товару:").grid(row=1, column=0,
padx=10, pady=10)
product_name_entry = tk.Entry(scrollable_frame)
product_name_entry.grid(row=1, column=1, padx=10, pady=10)
product_name_entry.bind("<KeyPress>", lambda event: on_key_press(event,
product_name_entry))
product_name_entry.bind("<Button-3>", lambda event:
show_context_menu(event, product_name_entry))
tk.Label(scrollable_frame, text="Опис:").grid(row=2, column=0, padx=10,
pady=10)
product_description_entry = tk.Entry(scrollable_frame)
product_description_entry.grid(row=2, column=1, padx=10, pady=10)
product_description_entry.bind("<KeyPress>", lambda event: on_key_press(event,
product_description_entry))

```

```

        product_description_entry.bind("<Button-3>", lambda event:
show_context_menu(event, product_description_entry))

        tk.Label(scrollable_frame, text="Ціна:").grid(row=3, column=0, padx=10,
pady=10)

        product_price_entry = tk.Entry(scrollable_frame)
        product_price_entry.grid(row=3, column=1, padx=10, pady=10)

        product_price_entry.bind("<KeyPress>", lambda event: on_key_press(event,
product_price_entry))

        product_price_entry.bind("<Button-3>", lambda event: show_context_menu(event,
product_price_entry))

        tk.Label(scrollable_frame, text="Посилання на фото товару:").grid(row=4,
column=0, padx=10, pady=10)

        product_image_url_entry = tk.Entry(scrollable_frame)
        product_image_url_entry.grid(row=4, column=1, padx=10, pady=10)

        product_image_url_entry.bind("<KeyPress>", lambda event: on_key_press(event,
product_image_url_entry))

        product_image_url_entry.bind("<Button-3>", lambda event:
show_context_menu(event, product_image_url_entry))

        tk.Label(scrollable_frame, text="Посилання на фото замірів:").grid(row=5,
column=0, padx=10, pady=10)

        product_measure_image_url_entry = tk.Entry(scrollable_frame)
        product_measure_image_url_entry.grid(row=5, column=1, padx=10, pady=10)

        product_measure_image_url_entry.bind("<KeyPress>", lambda event:
on_key_press(event, product_measure_image_url_entry))

        product_measure_image_url_entry.bind("<Button-3>", lambda event:
show_context_menu(event, product_measure_image_url_entry))

        tk.Label(scrollable_frame, text="Оберіть постачальника:").grid(row=6,
column=0, padx=10, pady=10)

        supplier_combobox = ttk.Combobox(scrollable_frame, state="readonly")
        suppliers = get_suppliers(connect_to_db())

```

```
supplier_combobox['values'] = [supplier[1] for supplier in suppliers] # Тільки  
назви постачальників
```

```
supplier_combobox.grid(row=6, column=1, padx=10, pady=10)
```

```
tk.Label(scrollable_frame, text="Оберіть категорію:").grid(row=7, column=0,  
padx=10, pady=10)
```

```
category_combobox = ttk.Combobox(scrollable_frame, state="readonly")
```

```
categories = get_categories(connect_to_db()) # Викликаємо функцію для  
отримання категорій
```

```
category_combobox['values'] = categories # Заповнюємо категорії
```

```
category_combobox.grid(row=7, column=1, padx=10, pady=10)
```

```
# Вибір кольорів
```

```
tk.Label(scrollable_frame, text="Оберіть кольори:").grid(row=8, column=0,  
padx=10, pady=10)
```

```
colors_frame = tk.Frame(scrollable_frame)
```

```
colors_frame.grid(row=8, column=1, padx=10, pady=10)
```

```
color_vars = {}
```

```
colors = get_colors(connect_to_db())
```

```
for color in colors:
```

```
    var = tk.IntVar()
```

```
    checkbox = tk.Checkbutton(colors_frame, text=color, variable=var)
```

```
    checkbox.pack(anchor='w')
```

```
    color_vars[color] = var
```

```
# Вибір розмірів
```

```
tk.Label(scrollable_frame, text="Оберіть розміри:").grid(row=9, column=0,  
padx=10, pady=10)
```

```
sizes_frame = tk.Frame(scrollable_frame)
```

```
sizes_frame.grid(row=9, column=1, padx=10, pady=10)
```

```
size_vars = {}
```

```
sizes = get_sizes(connect_to_db())
```

for size in sizes:

```
var = tk.IntVar()
checkbox = tk.Checkbutton(sizes_frame, text=size, variable=var)
checkbox.pack(anchor='w')
size_vars[size] = var
```

Кнопка для збереження товару

```
save_button = tk.Button(scrollable_frame, text="Зберегти", command=lambda:
save_product(
    product_code_entry.get(),
    product_name_entry.get(),
    product_description_entry.get(),
    product_price_entry.get(),
    product_image_url_entry.get(),
    product_measure_image_url_entry.get(),
    color_vars,
    size_vars,
    supplier_combobox.get(),
    category_combobox.get()
))
save_button.grid(row=10, columnspan=2, pady=20)
```

Функція для збереження товару в базі даних

```
def save_product(code, name, description, price, image_url, measure_image_url,
color_vars, size_vars, supplier_name, category_name):
    try:
        conn = connect_to_db()
        cursor = conn.cursor()

        # Збереження товару у таблиці products_od
        cursor.execute(
```

```

        "INSERT INTO products_od (code, product_name, description, image_url,
categories_od_category_name, price) VALUES (%s, %s, %s, %s, %s, %s)",
        (code, name, description, image_url, category_name, price)
    )

```

```

# Збереження замірів у таблиці measures_od

```

```

cursor.execute(
    "INSERT INTO measures_od (measures_url, description, products_od_code)
VALUES (%s, %s, %s)",
    (measure_image_url, description, code)
)

```

```

# Отримання supplier_id за supplier_name

```

```

cursor.execute(
    "SELECT supplier_id FROM suppliers_od WHERE supplier_name = %s",
    (supplier_name,)
)
supplier_id = cursor.fetchone()[0]

```

```

# Отримання size_id за кожен обраний розмір

```

```

selected_sizes = [size_name for size_name, var in size_vars.items() if var.get()
== 1]
for size_name in selected_sizes:
    cursor.execute(
        "SELECT size_id FROM sizes_od WHERE size_name = %s",
        (size_name,)
    )
    size_id = cursor.fetchone()[0]

```

```

# Збереження інформації про наявність у таблиці inventory_od для
кожного розміру та кольору

```

```

        selected_colors = [color_name for color_name, var in color_vars.items() if
var.get() == 1]
        for color_name in selected_colors:
            cursor.execute(
                "INSERT INTO inventory_od (products_od_code, available,
sizes_od_size_id, colors_od_colormame, suppliers_od_supplier_id) VALUES (%s, %s,
%s, %s, %s)",
                (code, True, size_id, color_name, supplier_id)
            )
            conn.commit()
            messagebox.showinfo("Успіх", "Товар успішно збережено!")
except Exception as e:
    conn.rollback()
    messagebox.showerror("Помилка збереження товару", str(e))
finally:
    cursor.close()
    conn.close()

```

Отримати дані з бази

```

def get_combined_data():
    conn = connect_to_db()
    cursor = conn.cursor()
    query = """
SELECT p.code, p.product_name, p.description, p.price, c.category_name,
       i.available, s.size_name, co.colormame, sup.supplier_name
FROM products_od p
JOIN categories_od c ON p.categories_od_category_name = c.category_name
JOIN inventory_od i ON i.products_od_code = p.code
JOIN sizes_od s ON i.sizes_od_size_id = s.size_id
JOIN colors_od co ON i.colors_od_colormame = co.colormame
JOIN suppliers_od sup ON i.suppliers_od_supplier_id = sup.supplier_id;

```

```
"""
```

```
cursor.execute(query)
```

```
data = cursor.fetchall()
```

```
conn.close()
```

```
return data
```

```
# Генерація звіту у форматі Word
```

```
def generate_word_report():
```

```
    data = get_combined_data()
```

```
        file_path = filedialog.asksaveasfilename(defaultextension=".docx",
```

```
filetypes=[("Word documents", "*.docx")])
```

```
    if file_path:
```

```
        document = Document()
```

```
        document.add_heading('Звіт з бази даних', 0)
```

```
    # Додаємо таблицю
```

```
    table = document.add_table(rows=1, cols=9)
```

```
        headers = ['Код', 'Назва товару', 'Опис', 'Ціна', 'Категорія', 'Доступність',  
'Розмір', 'Колір', 'Постачальник']
```

```
    hdr_cells = table.rows[0].cells
```

```
    for i, header in enumerate(headers):
```

```
        hdr_cells[i].text = header
```

```
    # Заповнюємо дані
```

```
    for row in data:
```

```
        row_cells = table.add_row().cells
```

```
        for i, value in enumerate(row):
```

```
            row_cells[i].text = str(value)
```

```
    document.save(file_path)
```

```
    messagebox.showinfo("Успіх", f"Звіт збережено як {file_path}")
```

```

# Генерація звіту у форматі Excel
def generate_excel_report():
    data = get_combined_data()
    file_path = filedialog.asksaveasfilename(defaultextension=".xlsx",
filetypes=[("Excel files", "*.xlsx")])
    if file_path:
        workbook = Workbook()
        sheet = workbook.active
        sheet.title = "Звіт"

        # Додаємо заголовки
        headers = ['Код', 'Назва товару', 'Опис', 'Ціна', 'Категорія', 'Доступність',
'Розмір', 'Колір', 'Постачальник']
        sheet.append(headers)

        # Додаємо дані
        for row in data:
            sheet.append(row)
        workbook.save(file_path)
        messagebox.showinfo("Успіх", f"Звіт збережено як {file_path}")

# Відкриваємо вікно для вибору формату звіту
def open_report_window():
    report_window = tk.Toplevel(root)
    report_window.title("Створити звіт")
    report_window.geometry("300x200")
    format_label = tk.Label(report_window, text="Оберіть формат звіту:")
    format_label.pack(pady=10)
    format_var = tk.StringVar(value="Word")
    word_option = ttk.Radiobutton(report_window, text="Word", variable=format_var,
value="Word")

```

```

        excel_option = ttk.Radiobutton(report_window, text="Excel",
variable=format_var, value="Excel")
    word_option.pack(pady=5)
    excel_option.pack(pady=5)

    generate_button = ttk.Button(report_window, text="Згенерувати",
command=lambda: generate_report(format_var.get()))
    generate_button.pack(pady=20)

# Генерація звіту на основі вибору
def generate_report(format_choice):
    if format_choice == "Word":
        generate_word_report()
    elif format_choice == "Excel":
        generate_excel_report()
def update_product():

# Створення нового вікна для оновлення даних
update_window = Toplevel(root)
update_window.title("Оновити дані товару")

# Підключення до бази для отримання розмірів і кольорів
try:
    conn = connect_to_db()
    cursor = conn.cursor()

    # Отримуємо список доступних розмірів
    cursor.execute("SELECT size_name FROM sizes_od;")
    sizes = [row[0] for row in cursor.fetchall()]

    # Отримуємо список доступних кольорів
    cursor.execute("SELECT colorname FROM colors_od;")

```

```

        colors = [row[0] for row in cursor.fetchall()]
except Exception as e:
    messagebox.showerror("Помилка", str(e))
    return

```

```

# Поля для введення даних

```

```

Label(update_window, text="Код товару:").grid(row=0, column=0)

```

```

product_code_entry = Entry(update_window)

```

```

product_code_entry.grid(row=0, column=1)

```

```

        Label(update_window, text="Нова кількість у наявності:").grid(row=1,
column=0)

```

```

available_entry = Entry(update_window)

```

```

available_entry.grid(row=1, column=1)

```

```

Label(update_window, text="Новий розмір:").grid(row=2, column=0)

```

```

size_combobox = ttk.Combobox(update_window, values=sizes)

```

```

size_combobox.grid(row=2, column=1)

```

```

Label(update_window, text="Новий колір:").grid(row=3, column=0)

```

```

color_combobox = ttk.Combobox(update_window, values=colors)

```

```

color_combobox.grid(row=3, column=1)

```

```

def submit_update():

```

```

    product_code = product_code_entry.get()

```

```

    available = available_entry.get()

```

```

    size = size_combobox.get()

```

```

    color = color_combobox.get()

```

```

    if not product_code:

```

```

        messagebox.showerror("Помилка", "Код товару не може бути порожнім")

```

```

        return

```

```

    try:

```

```

        cursor = conn.cursor()

```

```

# Оновлення кількості товару

```

if available:

```
cursor.execute("""
    UPDATE inventory_od
    SET available = %s
    WHERE products_od_code = %s;
""", (available, product_code))
```

Оновлення розміру товару

if size:

```
cursor.execute("""
    UPDATE inventory_od
    SET sizes_od_size_id = (SELECT size_id FROM sizes_od WHERE
size_name = %s)
    WHERE products_od_code = %s;
""", (size, product_code))
```

Оновлення кольору товару

if color:

```
cursor.execute("""
    UPDATE inventory_od
    SET colors_od_colorname = %s
    WHERE products_od_code = %s;
""", (color, product_code))
conn.commit()
messagebox.showinfo("Успіх", "Дані успішно оновлено!")
```

except Exception as e:

```
messagebox.showerror("Помилка", str(e))
```

finally:

```
cursor.close()
conn.close()
```

Кнопка для підтвердження оновлення

```

submit_button = Button(update_window, text="Оновити",
command=submit_update)

submit_button.grid(row=4, column=0, columnspan=2)

# Створення головного вікна програми
root = tk.Tk()
root.title("Система управління товарами")

# Рамка для вибору параметру пошуку
frame = tk.Frame(root)
frame.pack(pady=10)

# Поле для вибору типу пошуку (за кодом, категорією, розміром, кольором або
замірами)
search_by = tk.StringVar(value="code")
tk.Radiobutton(frame, text="Пошук за артикулом", variable=search_by,
value="code").pack(side=tk.LEFT, padx=5)
tk.Radiobutton(frame, text="Пошук за категорією", variable=search_by,
value="category").pack(side=tk.LEFT, padx=5)
tk.Radiobutton(frame, text="Пошук за розміром", variable=search_by,
value="size").pack(side=tk.LEFT, padx=5)
tk.Radiobutton(frame, text="Пошук за кольором", variable=search_by,
value="color").pack(side=tk.LEFT, padx=5)
tk.Radiobutton(frame, text="Пошук замірів", variable=search_by,
value="measures").pack(side=tk.LEFT, padx=5)

# Поле введення для значення пошуку
input_label = tk.Label(root, text="Введіть значення:")
input_label.pack(pady=5)
input_field = tk.Entry(root)
input_field.pack(pady=5)

```

```

# Кнопка для запуску пошуку
search_button = tk.Button(root, text="Пошук", command=search)
search_button.pack(pady=10)

# Кнопка "Додати товар"
add_product_button = tk.Button(root, text="Додати товар",
command=open_add_product_window)
add_product_button.pack(pady=10)

# Кнопка для створення звіту
report_button = ttk.Button(root, text="Створити звіт",
command=open_report_window)
report_button.pack(pady=50)
update_button = Button(root, text="Оновити товар", command=update_product)
update_button.pack()

# Створення рамки з прокруткою
scroll_canvas = tk.Canvas(root)
scroll_frame = tk.Frame(scroll_canvas)
scrollbar = tk.Scrollbar(root, orient="vertical", command=scroll_canvas.yview)
scroll_canvas.configure(yscrollcommand=scrollbar.set)
scrollbar.pack(side=tk.RIGHT, fill=tk.Y)
scroll_canvas.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
scroll_canvas.create_window((0, 0), window=scroll_frame, anchor="nw")
scroll_frame.bind("<Configure>", lambda event:
scroll_canvas.configure(scrollregion=scroll_canvas.bbox("all")))
# Прив'язка події прокрутки для різних операційних систем
scroll_canvas.bind_all("<MouseWheel>", on_mouse_wheel)
scroll_canvas.bind_all("<Button-4>", lambda event: scroll_canvas.yview_scroll(-1,
"units")) # Linux

```

```
scroll_canvas.bind_all("<Button-5>", lambda event: scroll_canvas.yview_scroll(1,  
"units")) # Linux
```

```
# Рамка для відображення результатів пошуку
```

```
result_frame = tk.Frame(scroll_frame)
```

```
result_frame.pack(pady=10)
```

```
# Запуск головного циклу
```

```
root.mainloop()
```